

G3T Up! Gravity Aligned Coordinate Frames Simplify Pointmap Processing

Bharath Raj Nagoor Kani Noah Snavely
Cornell University

Project Page: <https://g3t-paper.github.io/>

Abstract

Modern feed-forward 3D reconstruction methods like VGGT predict pixel-aligned pointmaps in camera-centric coordinate frames. However, this choice of coordinate frame is not always optimal. We propose instead to predict pointmaps in upright, gravity-aligned frames that exploit strong structural cues present in many real-world scenes. Unlike camera-centric frames, gravity-aligned frames share a common vertical axis across viewpoints, reducing the rotational degrees of freedom needed to relate pointmaps to one another. To this end, we introduce the Gravity Grounded Geometry Transformer (G3T), fine-tuned from existing models on gravity-aligned 3D data. G3T produces highly accurate gravity-aware predictions, including upright pointmaps and camera-to-gravity poses. We further introduce G3T-Long, a submap-based incremental 3D reconstruction pipeline that leverages the reduced rotational degrees of freedom afforded by upright frames to achieve significantly improved reconstruction accuracy.

1 Introduction

Pointmaps have recently become a widespread choice for representing image-centric 3D geometry in learned models. Popularized by DUST3R [45], they have become a standard representation in systems that perform feedforward 3D reconstruction [43, 47, 14, 21, 50].

A key design choice for a pointmap representation is *what coordinate frame it is defined in*. The default choice of popular methods is to represent predicted pointmaps in the coordinate frame of the first camera. While this is convenient and enables large-scale training on diverse datasets, it is not the only choice, and perhaps not the best choice for all tasks.

If the end goal is to create a 3D reconstruction of a scene from a set of images, all that matters is that the model can robustly reason about relative transformation between many pointmaps, and hence there is no restriction that we must use *camera* coordinate frames.

A good coordinate frame for joint 3D alignment across many views might be one that is aligned to strong structural regularities observed in a scene—that is, a world-centric, rather than camera-centric coordinate frame. Typical camera-centric 3D reconstruction methods like VGGT [43] involve a strong model asymmetry where the first image gets to define the coordinate frame, involving whatever arbitrary camera orientation that image might possess (including both camera pitch and roll). If instead the set of input images could agree on some natural world-centric coordinate frame, then this asymmetry would be resolved, potentially making for more stable, robust predictions. Unfortunately, it is difficult to define what a consistent world-centered coordinate frame would be.

In this work, we propose **Gravity Grounded Geometry Transformer (G3T)**, a feed-forward 3D prediction model that builds upon VGGT [43] and predicts pointmaps in *gravity-aligned* coordinate frames, exploiting strong structural regularities induced by gravity (see Figure 1).

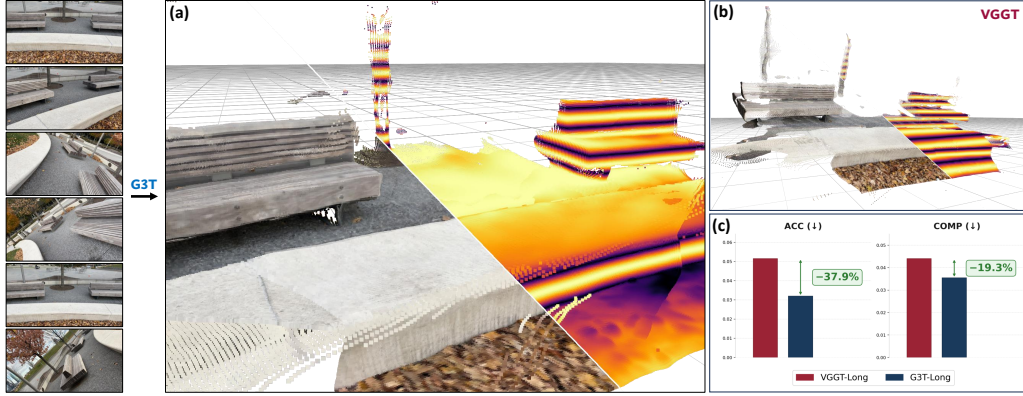


Figure 1: **Gravity Grounded Geometry Transformer (G3T)** predicts pointmaps aligned with scene gravity, leveraging structural cues inherent in natural scenes. We visualize uprightness using a ground-parallel grid and height-dependent color encoding. Pointmaps produced by G3T show near-constant color in *ground-parallel regions* (such as floors, benches etc.) indicating upright-alignment (a), whereas those produced by VGGT show rapidly varying colors in the same regions (b). Exploiting the reduced rotational degrees of freedom afforded by upright frames, we introduce G3T-Long: a robust, submap-based incremental 3D reconstruction pipeline. Results in (c) report median structure metrics averaged over 10 sequences from the TUM RGBD dataset (see Table 4 for full results).

Pointmaps predicted in such upright frames can be related to each other by a reduced 1 degree of freedom (1-DoF) rotation about the vertical axis (see Figure 2). The resulting predictions are natively defined in a natural, upright coordinate frame, making them immediately suitable for downstream tasks such as visualization or simulation without extra post-processing. We show that G3T yields camera poses and pointmaps that are more accurately aligned to gravity compared to baseline methods.

We further leverage these structured coordinate frames to develop **G3T-Long**, a submap-based incremental 3D reconstruction pipeline that extends VGGT-Long [8]. By restricting alignment to the reduced rotational degrees of freedom afforded by upright frames, G3T-Long achieves significantly more accurate reconstructions with reduced drift. While we instantiate these ideas on top of VGGT and VGGT-Long, the concept of using gravity-aligned coordinate frames is general and can be applied to other camera-centric methods as well.

2 Related work

Multi-view 3D pointmap prediction. DUST3R [45] introduced a transformer-based architecture for pixel-aligned pointmap prediction from unposed image pairs. Follow-up work extended it to predict in metric space [19], handle dynamic scenes [51, 12, 6, 9], and process input in a streaming fashion [42, 44, 7]. More recent work has shifted toward direct feedforward prediction from multi-view images [43, 35, 50, 21, 14] with subsequent work extending these methods to process long sequences [8, 23, 52, 11]. Despite these advances, little attention has been paid to the choice of coordinate frame. Of note, while π^3 [47] achieves invariance to input ordering, predictions remain in an arbitrary frame that does not exploit natural scene priors. Our focus is to explore the use of alternate, more world-centric coordinate frames for pointmap prediction.

Scene-level canonical frames from image level cues. Images often possess rich geometric cues such as straight lines, vanishing points that indicate dominant scene orientations, and objects that are upright or resting on flat surfaces. Classic work has sought to detect vanishing points from single images (e.g., by grouping detected lines) to aid in geometric calibration [33, 36, 5, 18]. More recently, learning-based approaches [49, 13, 39] were proposed to estimate gravity direction (and other properties) in a data-driven fashion. Other related work [41, 17] has explored *object-centric* coordinate frames, taking advantage of the fact that many object categories (cars, chairs, airplanes, etc) define natural coordinate frames. However, general scenes often do not have a single, unambiguous

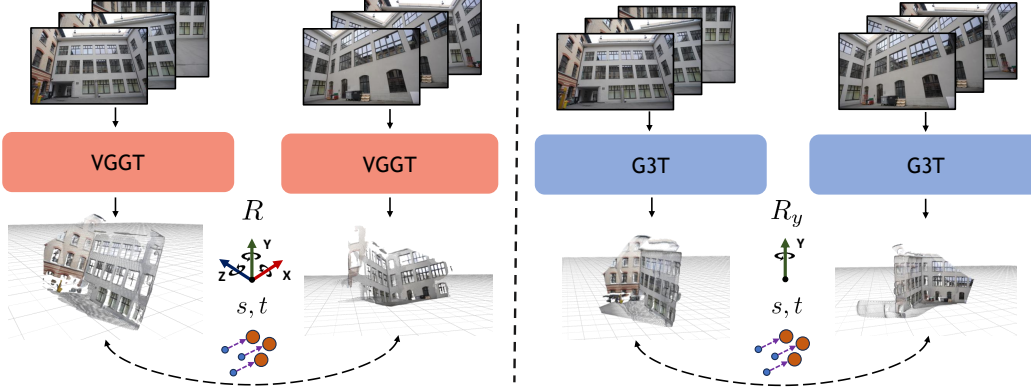


Figure 2: **Camera-aligned and gravity-aligned coordinate frames.** Feedforward 3D reconstruction methods such as VGGT [43] typically predict pointmaps in a camera coordinate frame. Such pointmaps can be related to each other using a pose $\pi(s, R, t) \in \text{Sim}(3)$ that has 7 degrees of freedom (DoF). In contrast, G3T predicts pointmaps in a gravity-aligned coordinate frame. Such pointmaps can be related to each other using a pose $\pi^y(s, R_y, t) \in \text{Sim}_y(3)$ that has 5-DoF (i.e., rotations are restricted to be rotations along the y -axis, which have 1-DoF). We leverage this property to constrain submap-based reconstruction algorithms (as described in Section 3.3).

world coordinate frame—but every scene on Earth has a gravity direction, making gravity-aligned coordinate frames an attractive choice for pointmap prediction tasks.

Leveraging gravity direction as a prior. Gravity direction has been leveraged as a prior in numerous vision tasks such as camera parameter estimation [16, 27], rotation averaging [24], and human motion reconstruction [31]. To the best of our knowledge, leveraging gravity direction in the context of pointmap prediction has not been previously explored. In fact, our work is motivated in part by the fact that, once a scene is in a more structured coordinate frame like those produced by our method, other tasks such as stereo estimation can benefit from stronger priors (see, for instance, work like Manhattan world stereo [10] and schematic surface reconstruction [48]).

3 Method

3.1 Preliminaries: pointmap prediction in camera frames

Given a set of N input images $\mathcal{I} = \{I_i\}$, VGGT [43] predicts: 1) a set of pixel-aligned pointmaps $\mathcal{X}^{C_1} = \{X_i^{C_1}\}$ and associated confidence maps $\Psi_{\mathcal{X}^{C_1}} = \{\psi_{X_i^{C_1}}\}$ (where $I_i, X_i^{C_1} \in \mathbb{R}^{H \times W \times 3}$ and $\psi_{X_i^{C_1}} \in \mathbb{R}^{H \times W}$), 2) a set of depthmaps $\mathcal{D} = \{D_i\}$ and associated confidence maps $\Psi_{\mathcal{D}} = \{\psi_{D_i}\}$ (where $D_i, \psi_{D_i} \in \mathbb{R}^{H \times W}$) and 3) camera parameters $\mathcal{G} = \{G_i\}$ (where $G_i \in \mathbb{R}^9$ is the concatenation of a rotation quaternion $q_i \in \mathbb{R}^4$, translation vector $t_i \in \mathbb{R}^3$ and field of view $f_i \in \mathbb{R}^2$). Note that the user has the choice of using either the predicted pointmaps directly, or using pointmaps created by unprojecting the depthmaps using the camera parameters.

3.2 Gravity-aligned pointmap prediction

Coordinate systems. A pointmap X^C in a camera coordinate frame can be related to a pointmap X^G in a *gravity-aligned* coordinate frame (that is, an upright frame) through the application of roll and pitch rotations such that the y -axis becomes gravity aligned (see Figure 2). In other words, $X^G = R_x R_z X^C$, where R_z and R_x represent suitable 3D roll and pitch rotations, respectively. While there are an infinite number of gravity-aligned coordinate frames, each related by a rotation about the up axis (the y -axis, in our case), the application of just roll and pitch (and not yaw) rotations uniquely determines a specific gravity-aligned coordinate frame for a given image (just as each image

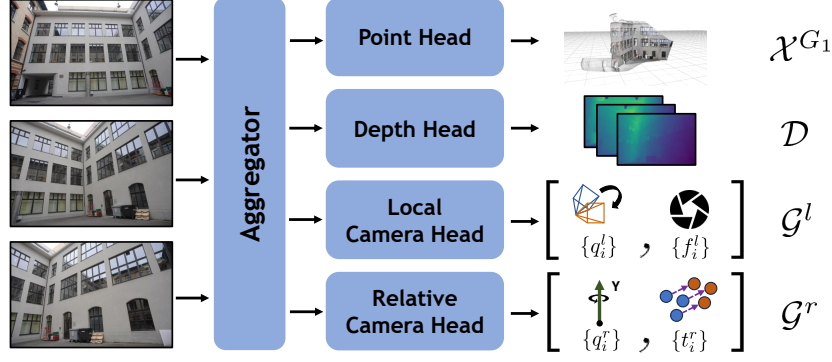


Figure 3: **Model architecture.** G3T builds upon VGGT with two key modifications. First, the point head outputs pointmaps in the *gravity-aligned* frame of the first image ($\mathcal{X}^{G_1} = \{X_i^{G_1}\}$). Second, we replace VGGT’s camera head with two new heads: the *local camera head*, whose outputs capture gravity-to-camera rotation and camera intrinsics parameters in $\mathcal{G}^l = \{G_i^l\}$; and the *relative camera head*, which capture 1-DoF relative yaw and translation parameters in $\mathcal{G}^r = \{G_i^r\}$. The aggregator, depth head, and point head architectures are otherwise unchanged from VGGT.

has its own camera frame). Note that the gravity-aligned coordinate frame has the same origin as that of the camera frame.

Pointmap prediction in gravity frames. We propose **G3T**, which builds upon VGGT with two key modifications (Figure 3). First, the point head now outputs pointmaps in the gravity frame of the first image (G_1) rather than its camera frame (C_1); we denote these as $\mathcal{X}^{G_1} = \{X_i^{G_1}\}$. Second, we replace VGGT’s camera head with two new heads: the *local camera head*, which outputs $\mathcal{G}^l = \{G_i^l\}$ where $G_i^l \in \mathbb{R}^6$ concatenates a rotation quaternion $q_i^l \in \mathbb{R}^4$ (gravity-to-camera rotation) and field of view $f_i^l \in \mathbb{R}^2$ (intrinsics); and the *relative camera head*, which outputs $\mathcal{G}^r = \{G_i^r\}$ where $G_i^r \in \mathbb{R}^5$ concatenates rotation parameters $q_i^r \in \mathbb{R}^2$ (1-DoF relative yaw w.r.t. the first frame) and translation $t_i^r \in \mathbb{R}^3$ (relative translation w.r.t. the first frame). The aggregator, depth head, and point head architectures are otherwise unchanged from VGGT. As in VGGT, gravity-aligned pointmaps can also be obtained by unprojecting depthmaps by using the composed local and relative poses.

3.3 Constraining submap-based incremental 3D reconstruction

Incremental 3D reconstruction via submap alignment. While VGGT can process an arbitrary number of images, in practice, the user is limited by GPU memory constraints. To this end, VGGT-Long [8] proposes splitting a sequence of images into overlapping chunks in a sliding window fashion and then aligning the chunk-wise pointmap predictions.

Suppose $\mathcal{A}^{C_1} = \{X_i^{C_1}\}$ and $\mathcal{B}^{C_2} = \{X_j^{C_2}\}$ are pointmap predictions obtained from two chunks with some overlapping images. Let $\mathcal{M}$ be the set of indices (i, j) that refer to the same image but in different chunks. Then, we can denote the set of corresponding pointmaps as $\{(X_i^{C_1}, X_j^{C_2})\}$, where $(i, j) \in \mathcal{M}$. Given this, the relative Sim(3) transformation π that aligns $\mathcal{B}^{C_2}$ to $\mathcal{A}^{C_1}$ can be computed using the following least squares objective:

$$\pi^* = \arg \min_{\pi \in \text{Sim}(3)} \sum_{(i,j) \in \mathcal{M}} \|X_i^{C_1} - \pi X_j^{C_2}\|_2^2 \quad (1)$$

This objective can be solved in closed form using Procrustes alignment [38]. In practice, VGGT-Long uses an iteratively re-weighted version of Eq. 1 (where the weights are initialized using confidence maps corresponding to the pointmaps; the interested reader can refer to their paper [8] for details). The relative transformations are computed between two types of chunks: 1) adjacent overlapping chunks, and 2) chunks constructed across loop closure detections. Once these are computed, VGGT-Long performs a global optimization over the absolute Sim(3) transformations of all chunks to be

Algorithm 1 <i>Procrustes</i> : Estimate $\pi(s, R, t)$ from pointmaps $\mathbf{P}, \mathbf{Q}$	Algorithm 2 <i>GA-Procrustes</i> : Estimate $\pi^y(s, R_y, t)$ from pointmaps $\mathbf{P}, \mathbf{Q}$
1: $\tilde{\mathbf{P}}, \tilde{\mathbf{Q}} \leftarrow \mathbf{P} - \mu_{\mathbf{P}}, \mathbf{Q} - \mu_{\mathbf{Q}}$	1: $\tilde{\mathbf{P}}, \tilde{\mathbf{Q}} \leftarrow \mathbf{P} - \mu_{\mathbf{P}}, \mathbf{Q} - \mu_{\mathbf{Q}}$
2: $s \leftarrow \ \tilde{\mathbf{Q}}\ _{\text{rms}} / \ \tilde{\mathbf{P}}\ _{\text{rms}}$	2: $s \leftarrow \ \tilde{\mathbf{Q}}\ _{\text{rms}} / \ \tilde{\mathbf{P}}\ _{\text{rms}}$
3: $\mathbf{H} \leftarrow (s\tilde{\mathbf{P}})^\top \tilde{\mathbf{Q}}$	3: $\tilde{\mathbf{P}}', \tilde{\mathbf{Q}}' \leftarrow \text{project}_{xz}(\tilde{\mathbf{P}}), \text{project}_{xz}(\tilde{\mathbf{Q}})$
4: $\mathbf{U}, \Sigma, \mathbf{V}^\top \leftarrow \text{SVD}(\mathbf{H})$	4: $\mathbf{H}' \leftarrow (s\tilde{\mathbf{P}}')^\top \tilde{\mathbf{Q}}'$
5: $\mathbf{R} \leftarrow \mathbf{V}\mathbf{U}^\top$	5: $\mathbf{U}', \Sigma', \mathbf{V}'^\top \leftarrow \text{SVD}(\mathbf{H}')$
6: $\mathbf{t} \leftarrow \mu_{\mathbf{Q}} - s\mathbf{R}\mu_{\mathbf{P}}$	6: $\mathbf{R}' \leftarrow \mathbf{V}'\mathbf{U}'^\top$
7: return $s, \mathbf{R}, \mathbf{t}$	7: $\mathbf{R}_y \leftarrow \text{lift}_{xyz}(\mathbf{R}')$
	8: $\mathbf{t} \leftarrow \mu_{\mathbf{Q}} - s\mathbf{R}_y\mu_{\mathbf{P}}$
	9: return $s, \mathbf{R}_y, \mathbf{t}$

consistent with these relative measurements. The optimization objective is (roughly) given by:

$$\{\pi_k^*\} = \arg \min_{\{\pi_k\}} \sum_{k=1}^{K-1} \|\delta_\pi(k, k+1)\|_2^2 + \sum_{(i,j) \in \mathcal{L}} \|\delta_\pi(i, j)\|_2^2 \quad (2)$$

Here, $\delta_\pi(i, j) = \tilde{\pi}_{i,j}^{-1} \pi_i^{-1} \pi_j$ measures the discrepancy between the relative pose $\tilde{\pi}_{i,j}$ estimated during chunk alignment and the relative pose derived by composing the absolute poses π_i, π_j being optimized. K denotes the number of overlapping chunks and $\mathcal{L}$ denotes the set of loop closure chunks. The norms in Equation 2 are applied to the Lie algebra representations of δ_π , as optimization is performed over $\mathfrak{sim}(3)$ (the 7-dimensional Lie algebra of $\text{Sim}(3)$) allowing the problem to be solved in an unconstrained manner using the Levenberg-Marquardt algorithm (see supplementary for more details)

Gravity-aligned Procrustes alignment. Given two pointmaps $X_i^{C_i}, X_i^{C_j}$ derived from the same image but represented in different coordinate frames C_i and C_j , we can estimate their relative pose using Procrustes [38] (Algorithm 1, where $\mathbf{P}, \mathbf{Q}$ are corresponding pointmaps, $\mu_{\mathbf{P}}, \mu_{\mathbf{Q}}$ their means, and $\|\cdot\|_{\text{rms}}$ the root mean squared distance). This yields a 7-DoF pose $\pi(s, R, t) \in \text{Sim}(3)$, with 3D rotation R , 3D translation t , and 1D scale s . We write $\pi = \mathcal{P}(X_i^{C_i}, X_i^{C_j})$ to denote the application of Procrustes.

Since gravity-aligned pointmaps are related by a 1-DoF rotation about the y -axis, we constrain Procrustes to a 2D rotation on the xz -plane, yielding our gravity aligned, i.e., *GA-Procrustes* algorithm (Algorithm 2, where project_{xz} and lift_{xyz} project to the xz -plane and reparameterize the result as a 3D y -axis rotation, respectively). GA-Procrustes produces a 5-DoF pose $\pi_y(s, R_y, t) \in \text{Sim}_y(3)$, where $\text{Sim}_y(3) \subset \text{Sim}(3)$ restricts rotations to the y -axis. We write $\pi_y = \mathcal{P}_{\text{GA}}(X_i^{G_i}, X_i^{G_j})$ to denote the application of GA-Procrustes.

Gravity-aligned incremental 3D reconstruction. We now describe how to construct G3T-Long, a gravity-aligned variant of VGGT-Long. The two key modifications are as follows. First, we replace Procrustes with GA-Procrustes in the chunk alignment procedure, which naturally constrains the estimated relative poses to $\text{Sim}_y(3)$. This gives the following modified chunk-alignment objective:

$$\pi^{y*} = \arg \min_{\pi^y \in \text{Sim}_y(3)} \sum_{(i,j) \in \mathcal{M}} \|X_i^{G_1} - \pi^y X_j^{G_2}\|_2^2 \quad (3)$$

Next, we redefine the global optimization over $\text{Sim}_y(3)$ rather than $\text{Sim}(3)$, with optimization performed over $\mathfrak{sim}_y(3)$ (the 5-dimensional Lie algebra of $\text{Sim}_y(3)$, reduced from the 7-dimensional $\mathfrak{sim}(3)$; see supplementary for more details). This gives us the following modified objective:

$$\{\pi_k^{y*}\} = \arg \min_{\{\pi_k^{y*}\}} \sum_{k=1}^{K-1} \|(\delta_{\pi^y}(k, k+1))\|_2^2 + \sum_{(i,j) \in \mathcal{L}} \|(\delta_{\pi^y}(i, j))\|_2^2 \quad (4)$$

All other implementation details remain unchanged.

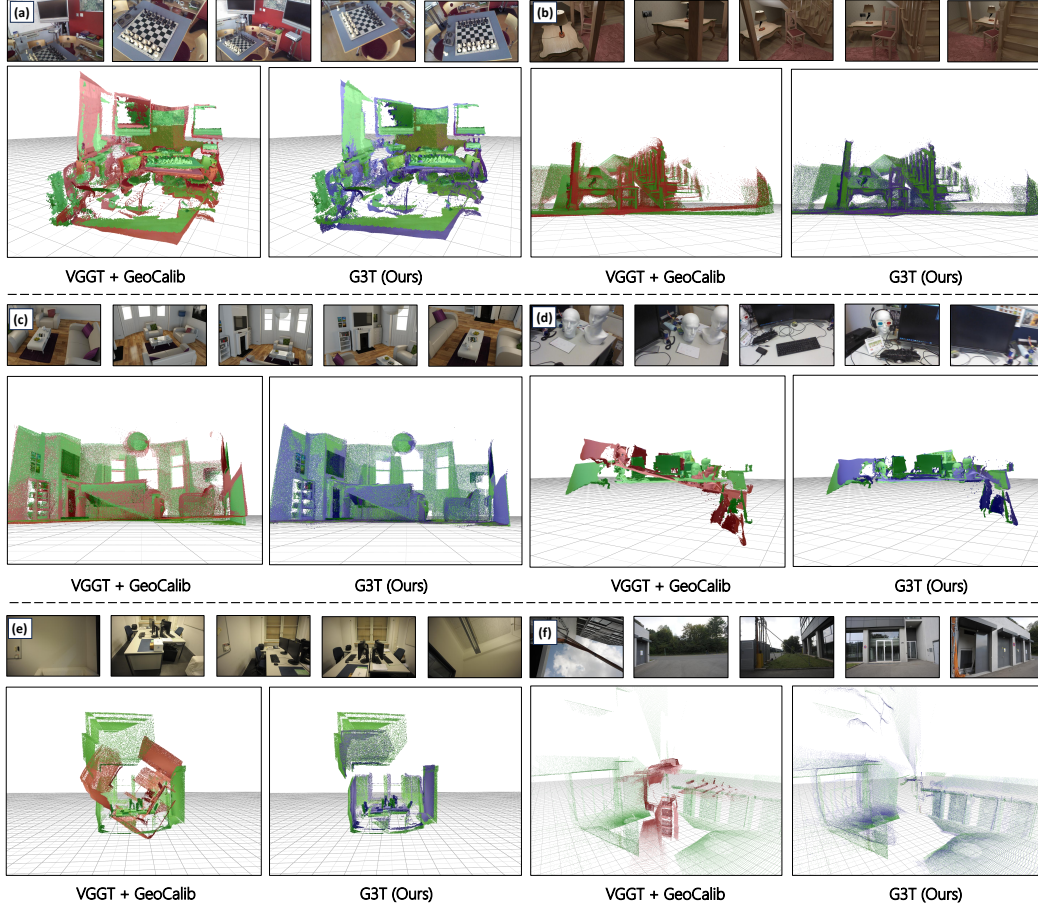


Figure 4: **G3T can consistently place pointmaps in an upright frame.** For the given set of input images, we compare VGGT predictions (**red**) made upright using GeoCalib, and our G3T predictions (**blue**) with ground-truth gravity-aligned pointmaps (**green**). We also render a grid depicting a plane parallel to the ground. We observe that the composition of VGGT and GeoCalib often produces “slanted” pointmap predictions that are misaligned with the vertical direction. In contrast, G3T produces pointmaps that are better aligned along the vertical direction, even in scenes captured in images with large pitch angles.

3.4 Training details

To create G3T, we fine-tune the open-source VGGT model on gravity-aligned pointmaps. However, the ground-truth information available found in popular 3D datasets are not necessarily gravity-aligned. Inspired by prior work [37, 39] that uses COLMAP’s `model_orientation_aligner` [29, 28] for Manhattan alignment, we apply it to gravity-align ground-truth for datasets that are not naturally gravity-aligned (see supplementary for details). Overall, we use gravity-aligned data from five large-scale datasets (MegaDepth [20], Hypersim [26], ARKitScenes [2], DL3DV [22] and TartanAir [46]) for fine-tuning.

We opt for fine-tuning the entire VGGT architecture (no weights are frozen). We use the same set of losses as VGGT for training, but with gravity-aligned ground truth points. We ignore the track head. Within each batch, we normalize predicted and ground-truth pointmaps independently before computing the loss (as done in DUST3R [45]). Our final model was fine-tuned on 8 A100 GPUs for 40 epochs, with 1000 data samples per epoch (which took approximately 1 week). For each batch, we randomly sample 2-12 views per scene for a maximum of 96 images per GPU. We use a gradient accumulation of 4 and a cosine learning rate schedule that starts at $1e-6$ after 1 warmup epoch. The remaining training configuration mimics the released version of VGGT.

Table 1: **Camera to gravity rotation estimation results.** We evaluate the quality of camera-to-gravity pose estimation when provided with 1, 4, or 8 input images on three datasets that were unseen during training. We observe that methods based on G3T (in particular Local Head) performs much better than GeoCalib across most metrics. Additionally, methods based on G3T show greater reduction in mean rotation error than GeoCalib. Rotation errors are given in degrees.

Dataset	Method	1V					4V					8V				
		$R_{err} (\downarrow)$		$R_{acc} (\uparrow)$			$R_{err} (\downarrow)$		$R_{acc} (\uparrow)$			$R_{err} (\downarrow)$		$R_{acc} (\uparrow)$		
		Mean	Median	@1°	@2°	@5°	Mean	Median	@1°	@2°	@5°	Mean	Median	@1°	@2°	@5°
7Scenes	GeoCalib	6.78	3.07	9.60	29.20	74.80	6.60	2.79	11.20	32.00	76.80	6.56	2.75	10.80	32.40	77.60
	Procrustes	2.00	1.73	15.60	60.40	96.40	1.87	1.59	23.60	63.60	97.20	1.88	1.64	22.00	61.60	97.20
	Local Head	1.92	1.68	18.00	64.40	96.80	1.78	1.54	26.00	66.80	98.00	1.78	1.49	26.00	65.20	98.40
NRGBD	GeoCalib	2.61	1.91	26.80	53.20	89.60	2.28	1.50	38.80	65.20	92.00	2.19	1.30	41.20	68.40	91.60
	Procrustes	1.47	0.71	60.40	78.40	93.60	1.32	0.60	70.00	78.00	95.60	1.26	0.50	71.20	78.80	95.20
	Local Head	1.33	0.63	65.60	78.80	94.00	1.21	0.46	76.80	80.40	95.20	1.13	0.40	79.20	80.40	94.00
ETH3D	GeoCalib	2.24	0.73	68.80	90.40	97.20	2.22	0.71	69.20	91.20	97.60	2.21	0.72	69.60	90.80	98.00
	Procrustes	1.96	1.08	44.00	78.80	98.00	1.95	1.00	50.00	80.40	95.60	1.85	1.03	47.60	80.40	96.00
	Local Head	1.62	0.60	79.20	95.60	98.40	1.11	0.55	81.20	97.60	98.80	1.05	0.57	81.60	98.00	98.80

Table 2: **Multi-view pointmap errors.** We compare the performance of VGGT and G3T on structure metrics on three datasets that were unseen during training. We observe that G3T retains similar structure performance while having the property of being upright (as observed in Table 1). Furthermore, the depthmap unprojection results indicate that the relative yaw predictions output by G3T are stable as well.

Model	7Scenes			NRGBD			ETH3D		
	ACC ($\downarrow$)	COMP ($\downarrow$)	NC ($\uparrow$)	ACC ($\downarrow$)	COMP ($\downarrow$)	NC ($\uparrow$)	ACC ($\downarrow$)	COMP ($\downarrow$)	NC ($\uparrow$)
VGGT _P	0.029	0.034	0.796	0.024	0.019	0.921	0.191	0.191	0.890
VGGT _D	0.031	0.032	0.753	0.022	0.018	0.913	0.209	0.174	0.880
G3T _P	0.028	0.032	0.793	0.026	0.021	0.907	0.188	0.181	0.892
G3T _D	0.026	0.029	0.780	0.026	0.021	0.900	0.194	0.165	0.882

4 Experiments

We begin our analysis by first evaluating the uprightness of the camera-to-gravity poses and pointmaps predicted by G3T (Section 4.1). Then, we analyze the effectiveness of performing gravity-aligned incremental 3D reconstruction using G3T (Section 4.2).

4.1 Evaluating uprightness

Evaluation setup. To evaluate the uprightness of G3T’s predictions, we estimate the camera-to-gravity rotation via two methods: (1) *Local Head*, which directly extracts the rotation from local camera head outputs $\mathcal{G}^l$, and (2) *Procrustes*, where Procrustes is used on the pointmap outputs of G3T and VGGT. As a baseline, we evaluate GeoCalib [39] (with multi-image optimization). Given that GeoCalib can be applied as a post-process to make VGGT upright, we are effectively testing this composed strategy against direct gravity-frame prediction by G3T. To assess the impact of fine-tuning on structure quality, we additionally compute accuracy (ACC), completeness (COMP), and normal consistency (NC) following prior work [44, 42, 43, 47], after aligning predicted and ground-truth pointmaps via Procrustes. Pointmaps are obtained either directly from model predictions (VGGT_P and G3T_P) or by unprojecting depth predictions using estimated cameras (VGGT_D and G3T_D).

For camera-to-gravity evaluation, we test with 1, 4, and 8 input views on 7Scenes [32], NRGBD [1], and ETH3D [30], all of which are unseen during training for both models. For structure evaluation, we use sets of five input views from the same datasets. We elaborate on the pre-processing and view sampling setup in the supplementary.

On using Procrustes. Given a set of N images $\mathcal{I}$, suppose we predict a set of gravity-aligned pointmaps $\mathcal{X}^{G_1}$ using G3T, and a set of corresponding camera-frame pointmaps $\mathcal{X}^{C_1}$ predicted using VGGT. We can then compute the camera-to-gravity rotation using Procrustes alignment between the two sets of pointmaps. Since pointmap predictions can be noisy, we implement a RANSAC loop which incorporates pointmap confidences to obtain a robust estimate of the rotation matrix (please refer to the supplementary for more details).

Table 3: **Submap-based incremental 3D reconstruction on TUM RGBD: Ambient frame pose metrics.** We measure the quality of the optimized absolute poses per chunk in each model’s *ambient* frame: camera frame for VGGT and gravity frame for G3T (as described in Section 4.2). N denotes the number of overlapping chunks a sequence was split into. We can see that G3T generally has superior performance in APE metrics. This indicates that leveraging gravity-aligned pointmap predictions and performing chunk alignment and optimization in $\text{Sim}_y(3)$ improves robustness. Additionally, we observe lesser drift along the y -axis when using G3T.

Method	Metric ($\downarrow$)	fr1/360 ($N=8$)	fr1/desk ($N=6$)	fr1/desk2 ($N=7$)	fr1/room ($N=15$)	fr1/plant ($N=13$)	fr1/teddy ($N=16$)	fr3/loh ¹ ($N=28$)	fr2/ps ¹ ($N=24$)	fr2/ps2 ¹ ($N=18$)	fr2/ps3 ¹ ($N=25$)
VGGT-Long	APE_R	16.320	2.313	1.553	4.276	2.963	4.732	4.289	5.924	13.845	15.228
	APE_t	0.050	0.025	0.037	0.179	0.053	0.083	0.326	0.444	0.553	0.947
	δ_y	0.018	0.005	0.009	0.029	0.018	0.045	0.160	0.224	0.358	0.368
G3T-Long	APE_R	19.309	1.434	5.192	3.502	1.429	1.948	3.964	3.482	3.512	6.381
	APE_t	0.056	0.012	0.031	0.178	0.036	0.056	0.170	0.255	0.235	0.220
	δ_y	0.009	0.008	0.015	0.033	0.016	0.016	0.023	0.032	0.032	0.029

Table 4: **Submap-based incremental 3D reconstruction on TUM RGBD: Structure metrics.** We measure frame-independent structure metrics to evaluate the reconstruction quality (as described in Section 4.2). N denotes the number of overlapping chunks a sequence was split into. We can see that G3T generally outperforms VGGT in most cases. These results complement those observed in Table 3, as they are a consequence of superior chunk alignment achieved in the gravity frame.

Method	Metric ($\downarrow$)	fr1/360 ($N=8$)	fr1/desk ($N=6$)	fr1/desk2 ($N=7$)	fr1/room ($N=15$)	fr1/plant ($N=13$)	fr1/teddy ($N=16$)	fr3/loh ¹ ($N=28$)	fr2/ps ¹ ($N=24$)	fr2/ps2 ¹ ($N=18$)	fr2/ps3 ¹ ($N=25$)
VGGT-Long	ACC ($\downarrow$)	0.057	0.009	0.014	0.046	0.032	0.028	0.064	0.111	0.196	0.272
	COMP ($\downarrow$)	0.071	0.012	0.016	0.043	0.041	0.044	0.044	0.048	0.140	0.201
	NC ($\uparrow$)	0.701	0.628	0.619	0.664	0.619	0.594	0.606	0.663	0.700	0.678
G3T-Long	ACC ($\downarrow$)	0.043	0.009	0.013	0.040	0.024	0.021	0.024	0.059	0.069	0.084
	COMP ($\downarrow$)	0.058	0.011	0.015	0.035	0.033	0.036	0.021	0.036	0.067	0.068
	NC ($\uparrow$)	0.714	0.616	0.606	0.662	0.621	0.602	0.647	0.705	0.731	0.748

Performance measures. We compare the estimated camera-to-gravity pose of the 1st frame to the corresponding ground-truth. While both the local head of G3T and GeoCalib can provide estimates for all frames, we only evaluate the 1st frame pose: 1) to be consistent with the Procrustes method which outputs the pose estimate only for the 1st frame, and 2) to cleanly measure the impact of adding more inputs views on resolving ambiguities in estimating gravity direction. We use the geodesic rotation error (as defined in [4]) to measure performance:

$$R_{\text{err}} = \arccos \left(\frac{\text{Trace}(R_2 R_1^T) - 1}{2} \right) \quad (5)$$

We also compute rotation accuracy by calculating the percentage of samples with rotation error less than $1/2/5$ degrees.

Observation 1: *G3T can estimate high-quality camera-to-gravity poses.* Quantitative results are shown in Table 1. We see that the rotation estimates obtained using G3T consistently outperform GeoCalib on most metrics. In particular, the mean error for 7Scenes and NRGBD is reduced by more than half. Also, we note that the rotation matrix extracted directly from the local head tends to be better than Procrustes. Finally, we note that methods based on G3T tends to have a greater reduction in rotation error when the number of views is increased (for example, see the mean rotation error for ETH3D and 7Scenes), thus pointing to the effectiveness of G3T incorporating structural cues from multi-view data.

Observation 2: *Pointmaps predicted by G3T have stronger gravity-alignment while retaining structure quality.* The Procrustes results in Table 1 are consistently lower than the corresponding GeoCalib values. This provides an indirect measure of pointmap uprightness. This improvement is supported by qualitative visualizations in Figure 4, where we can observe that G3T produces pointmaps with stronger gravity-alignment than those obtained by lifting VGGT predictions to the gravity frame using GeoCalib. Additionally, the comparable pointmap structure performance to VGGT in Table 2 confirms that upright alignment is achieved while retaining structure quality. The

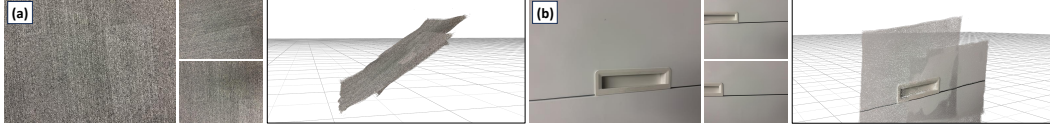


Figure 5: **Visualizing failure cases.** G3T struggles to resolve upright alignment in scenes with ambiguous structural cues. We illustrate this with two self-captured examples. In (a), close-up floor images cause G3T to produce a slanted pointmap. In (b), horizontally rotated images of a vertically-aligned cabinet lead to pointmaps with incorrect orientation.

depthmap unprojection results additionally validate that the relative yaws predicted by the global head are informative.

4.2 Submap-based incremental 3D reconstruction

Evaluation setup. We evaluate two quantities to measure the effectiveness of gravity-aligned submap-based incremental reconstruction. First, we measure the quality of the optimized absolute poses per chunk against ground-truth poses in their respective *ambient* frame. In other words, the ground-truth poses are camera-to-world poses for VGGT (which operates in $\text{Sim}(3)$) and are gravity-to-world poses for G3T (which operates in $\text{Sim}_y(3)$). Second, we measure overall structure quality after incremental reconstruction. For both experiments, we use pointmaps obtained from the point head.

We evaluate on sequences selected from the TUM RGBD dataset [34]. As described in Section 3.3, sequences are broken into overlapping chunks and then processed in a sliding window fashion. We set the maximum chunk size to 25 frames, with 7 overlapping frames between adjacent chunks. Further details on preprocessing, hyperparameters, and analyses (including the impact of different hyperparameter choices and usage of depthmap-unprojected points) are provided in the supplementary.

Performance measures. For pose evaluation, we report the RMSE of the absolute pose error for rotation (APE_R , degrees) and translation (APE_t , metres), along with the median absolute error of the y -axis (vertical) translation component (δ_y , metres). For structure quality, we use the structure metrics from Section 4.1

Observation 1: *G3T-Long has lower ambient frame pose errors.* From Table 3, we can see that, in most cases, performing incremental 3D reconstruction using G3T-Long leads to better performance on APE_R and APE_t . This suggests that performing 1-DoF rotation constrained incremental 3D reconstruction using gravity-aligned pointmaps is more robust. Additionally, we observe that G3T generally has small δ_y values, unlike VGGT-Long which has high variance across scenes. Since the upward direction is aligned with the y -axis for G3T, this implies that our gravity-aligned incremental reconstruction procedure reduces vertical drift.

Observation 2: *G3T-Long has better structure metrics.* From Table 3, we can see that G3T-Long has significant improvements over VGGT-Long on structure metrics. These results complement those observed in Table 3, as they follow from superior chunk alignment achieved in the gravity frame.

5 Discussion

Our work highlights an underexplored aspect of 3D pointmap prediction models—the choice of coordinate frame—and argues for the use frames with more structure, in particular *gravity-aligned frames*. Beyond producing accurate upright-aware predictions, we demonstrate that we can perform robust submap-based incremental 3D reconstruction by leveraging the reduced degrees of freedom of gravity-aligned frames. While our work takes a step towards consideration of structured coordinate frames, we note some limitations and directions for future work.

Limitations. G3T may not produce good upright-aware predictions in scenes with ambiguous structural cues. For example, G3T can struggle to estimate upright pointmaps from close-up images of floors and walls if additional unambiguous context is not present (see Figure 5).

¹We shorten `long_office_household` to `loh` and `pioneer_slam` to `ps` for brevity.

Future work. There are many exciting directions for future work. For instance, it would be interesting to explore if gravity-aligned prediction naturally encourages performance improvement in tasks that favor uprightedness, such as physically based simulation or spatial reasoning. Additionally, it could also be advantageous to explore coordinate frames with even more structure, such as Manhattan frames (when appropriate), or even compositions of coordinate frames (e.g., a mixture of an overall scene coordinate frame, as well as local object coordinate frames for items in the scene).

Acknowledgements

We thank the authors of DUST3R [45], VGGT [43], CUT3R [44], and VGGT-Long [8] for open-sourcing their projects, which our work builds upon. Additionally, we would like to thank Aditya Chetan, Haian Jin and Jay Karhade for their feedback on initial drafts of the paper. This work was funded in part by the National Science Foundation (IIS-2211259 and IIS-2212084), and benefited from the NVIDIA Academic Grant Program for compute resources.

References

- [1] Dejan Azinović, Ricardo Martin-Brualla, Dan B Goldman, Matthias Nießner, and Justus Thies. Neural rgb-d surface reconstruction. In *CVPR*, pages 6290–6301, June 2022.
- [2] Gilad Baruch, Zhuoyuan Chen, Afshin Dehghan, Tal Dimry, Yuri Feigin, Peter Fu, Thomas Gebauer, Brandon Joffe, Daniel Kurz, Arik Schwartz, and Elad Shulman. ARKitscenes - a diverse real-world dataset for 3d indoor scene understanding using mobile RGB-d data. In *NeurIPS Datasets and Benchmarks*, 2021.
- [3] Romain Brégier. Deep regression on manifolds: a 3D rotation case study. In *2021 International Conference on 3D Vision (3DV)*, 2021.
- [4] Ruojin Cai, Jason Y Zhang, Philipp Henzler, Zhengqi Li, Noah Snavely, and Ricardo Martin-Brualla. Can generative video models help pose estimation? In *CVPR*, 2025.
- [5] Bruno Caprile and Vincent Torre. Using vanishing points for camera calibration. In *IJCV*, 1990.
- [6] Xingyu Chen, Yue Chen, Yuliang Xiu, Andreas Geiger, and Anpei Chen. Easi3r: Estimating disentangled motion from dust3r without training. In *ICCV*, 2025.
- [7] Xingyu Chen, Yue Chen, Yuliang Xiu, Andreas Geiger, and Anpei Chen. Ttt3r: 3d reconstruction as test-time training. *ICLR*, 2026.
- [8] Kai Deng, Zexin Ti, Jiawei Xu, Jian Yang, and Jin Xie. Vggt-long: Chunk it, loop it, align it – pushing vggt’s limits on kilometer-scale long rgb sequences. *arXiv preprint arXiv:2507.16443*, 2025.
- [9] Haiwen Feng*, Junyi Zhang*, Qianqian Wang, Yufei Ye, Pengcheng Yu, Michael J. Black, Trevor Darrell, and Angjoo Kanazawa. St4rtrack: Simultaneous 4d reconstruction and tracking in the world. In *ICCV*, 2025.
- [10] Yasutaka Furukawa, Brian Curless, Steven M. Seitz, and Richard Szeliski. Manhattan-world stereo. In *CVPR*, 2009.
- [11] Haian Jin, Rundi Wu, Tianyuan Zhang, Ruiqi Gao, Jonathan T. Barron, Noah Snavely, and Aleksander Holynski. ZipMap: Linear-time stateful 3d reconstruction via test-time training. In *CVPR*, 2026.
- [12] Linyi Jin, Richard Tucker, Zhengqi Li, David Fouhey, Noah Snavely, and Aleksander Holynski. Stereo4d: Learning how things move in 3d from internet stereo videos. In *CVPR*, 2025.
- [13] Linyi Jin, Jianming Zhang, Yannick Hold-Geoffroy, Oliver Wang, Kevin Matzen, Matthew Sticha, and David F. Fouhey. Perspective fields for single image camera calibration. In *CVPR*, 2023.

- [14] Nikhil Keetha, Norman Müller, Johannes Schönberger, Lorenzo Porzi, Yuchen Zhang, Tobias Fischer, Arno Knapitsch, Duncan Zauss, Ethan Weber, Nelson Antunes, Jonathon Luiten, Manuel Lopez-Antequera, Samuel Rota Bulò, Christian Richardt, Deva Ramanan, Sebastian Scherer, and Peter Kotschieder. MapAnything: Universal feed-forward metric 3D reconstruction. In *International Conference on 3D Vision (3DV)*, 2026.
- [15] Jatavallabhula Krishna Murthy, Soroush Saryazdi, Ganesh Iyer, and Liam Paull. gradslam: Dense slam meets automatic differentiation. In *IEEE International Conference on Robotics & Automation (ICRA)*, 2020.
- [16] Zuzana Kukelova, Martin Bujnak, and Tomas Pajdla. Closed-form solutions to minimal absolute pose problems with known vertical direction. In *ACCV*, 2010.
- [17] Nilesh Kulkarni, Abhinav Gupta, and Shubham Tulsiani. Canonical surface mapping via geometric cycle consistency. In *ICCV*, 2019.
- [18] Hyunjoon Lee, Eli Shechtman, Jue Wang, and Seungyong Lee. Automatic upright adjustment of photographs with robust camera calibration. In *IEEE TPAMI*, 2014.
- [19] Vincent Leroy, Yohann Cabon, and Jérôme Revaud. Grounding image matching in 3d with mast3r. In *ECCV*, 2024.
- [20] Zhengqi Li and Noah Snavely. Megadepth: Learning single-view depth prediction from internet photos. In *CVPR*, 2018.
- [21] Haotong Lin, Sili Chen, Jun Hao Liew, Donny Y. Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. Depth anything 3: recovering the visual space from any views. *arXiv preprint arXiv:2511.10647*, 2025.
- [22] Lu Ling, Yichen Sheng, Zhi Tu, Wentian Zhao, Cheng Xin, Kun Wan, Lantao Yu, Qianyu Guo, Zixun Yu, Yawen Lu, et al. D13dv-10k: A large-scale scene dataset for deep learning-based 3d vision. In *CVPR*, pages 22160–22169, 2024.
- [23] Dominic Maggio, Hyungtae Lim, and Luca Carlone. Vggt-slam: Dense rgb slam optimized on the sl (4) manifold. In *NeurIPS*, 2025.
- [24] Linfei Pan, Marc Pollefeys, and Dániel Baráth. Gravity-aligned rotation averaging with circular regression. In *ECCV*, 2024.
- [25] Zador Pataki, Paul-Edouard Sarlin, Johannes L. Schönberger, and Marc Pollefeys. MP-SfM: Monocular Surface Priors for Robust Structure-from-Motion. In *CVPR*, 2025.
- [26] Mike Roberts, Jason Ramapuram, Anurag Ranjan, Atulit Kumar, Miguel Angel Bautista, Nathan Paczan, Russ Webb, and Joshua M. Susskind. Hypersim: A photorealistic synthetic dataset for holistic indoor scene understanding. In *ICCV*, 2021.
- [27] Olivier Saurer, Pascal Vasseur, Rémi Boutteau, Cédric Demonceaux, Marc Pollefeys, and Friedrich Fraundorfer. Homography based egomotion estimation with a common direction. *IEEE TPAMI*, 2017.
- [28] Johannes Lutz Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [29] Johannes Lutz Schönberger, Enliang Zheng, Marc Pollefeys, and Jan-Michael Frahm. Pixelwise view selection for unstructured multi-view stereo. In *European Conference on Computer Vision (ECCV)*, 2016.
- [30] Thomas Schöps, Johannes L. Schönberger, Silvano Galliani, Torsten Sattler, Konrad Schindler, Marc Pollefeys, and Andreas Geiger. A multi-view stereo benchmark with high-resolution images and multi-camera videos. In *CVPR*, 2017.
- [31] Zehong Shen, Huaijin Pi, Yan Xia, Zhi Cen, Sida Peng, Zechen Hu, Hujun Bao, Ruizhen Hu, and Xiaowei Zhou. World-grounded human motion recovery via gravity-view coordinates. In *SIGGRAPH Asia Conference Proceedings*, 2024.

- [32] Jamie Shotton, Ben Glocker, Christopher Zach, Shahram Izadi, Antonio Criminisi, and Andrew Fitzgibbon. Scene coordinate regression forests for camera relocalization in rgb-d images. In *CVPR*, June 2013.
- [33] Jefferey A. Shufelt. Performance evaluation and analysis of vanishing point detection techniques. *IEEE TPAMI*, 21(3):282–288, 1999.
- [34] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers. A benchmark for the evaluation of rgb-d slam systems. In *Proc. of the Int. Conf. on Intelligent Robot Systems (IROS)*, 2012.
- [35] Zhenggang Tang, Yuchen Fan, Dilin Wang, Hongyu Xu, Rakesh Ranjan, Alexander Schwing, and Zhicheng Yan. Mv-dust3r+: Single-stage scene reconstruction from sparse views in 2 seconds. In *CVPR*, 2025.
- [36] Jean-Philippe Tardif. Non-iterative approach for fast and accurate vanishing point detection. In *ICCV*, 2009.
- [37] Joseph Tung, Gene Chou, Ruojin Cai, Guandao Yang, Kai Zhang, Gordon Wetzstein, Bharath Hariharan, and Noah Snavely. Megascenes: Scene-level view synthesis at scale. In *ECCV*, 2024.
- [38] S. Umeyama. Least-squares estimation of transformation parameters between two point patterns. *IEEE TPAMI*, 1991.
- [39] Alexander Veicht, Paul-Edouard Sarlin, Philipp Lindenberger, and Marc Pollefeys. GeoCalib: Single-image Calibration with Geometric Optimization. In *ECCV*, 2024.
- [40] Chen Wang, Dasong Gao, Kuan Xu, Junyi Geng, Yaoyu Hu, Yuheng Qiu, Bowen Li, Fan Yang, Brady Moon, Abhinav Pandey, Aryan, Jiahe Xu, Tianhao Wu, Haonan He, Daning Huang, Zhongqiang Ren, Shibo Zhao, Taimeng Fu, Pranay Reddy, Xiao Lin, Wenshan Wang, Jingnan Shi, Rajat Talak, Kun Cao, Yi Du, Han Wang, Huai Yu, Shanzhao Wang, Siyu Chen, Ananth Kashyap, Rohan Bandaru, Karthik Dantu, Jiajun Wu, Lihua Xie, Luca Carlone, Marco Hutter, and Sebastian Scherer. PyPose: A library for robot learning with physics-based optimization. In *CVPR*, 2023.
- [41] He Wang, Srinath Sridhar, Jingwei Huang, Julien Valentin, Shuran Song, and Leonidas J. Guibas. Normalized object coordinate space for category-level 6d object pose and size estimation. In *CVPR*, 2019.
- [42] Hengyi Wang and Lourdes Agapito. 3d reconstruction with spatial memory. In *International Conference on 3D Vision (3DV)*, 2025.
- [43] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. Vggt: Visual geometry grounded transformer. In *CVPR*, 2025.
- [44] Qianqian Wang*, Yifei Zhang*, Aleksander Holynski, Alexei A. Efros, and Angjoo Kanazawa. Continuous 3d perception model with persistent state. In *CVPR*, 2025.
- [45] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jerome Revaud. Dust3r: Geometric 3d vision made easy. In *CVPR*, 2024.
- [46] Wenshan Wang, DeLong Zhu, Xiangwei Wang, Yaoyu Hu, Yuheng Qiu, Chen Wang, Yafei Hu, Ashish Kapoor, and Sebastian Scherer. Tartanair: A dataset to push the limits of visual slam. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [47] Yifan Wang, Jianjun Zhou, Haoyi Zhu, Wenzheng Chang, Yang Zhou, Zizun Li, Junyi Chen, Jiangmiao Pang, Chunhua Shen, and Tong He. π^3 : Permutation-equivariant visual geometry learning. *ICLR*, 2026.
- [48] Changchang Wu, Sameer Agarwal, Brian Curless, and Steven M. Seitz. Schematic surface reconstruction. In *CVPR*, 2012.
- [49] Wenqi Xian, Zhengqi Li, Matthew Fisher, Jonathan Eisenmann, Eli Shechtman, and Noah Snavely. Uprightnet: geometry-aware camera orientation estimation from single images. In *ICCV*, 2019.

- [50] Jianing Yang, Alexander Sax, Kevin J. Liang, Mikael Henaff, Hao Tang, Ang Cao, Joyce Chai, Franziska Meier, and Matt Feiszli. Fast3r: Towards 3d reconstruction of 1000+ images in one forward pass. In *CVPR*, 2025.
- [51] Junyi Zhang, Charles Herrmann, Junhwa Hur, Varun Jampani, Trevor Darrell, Forrester Cole, Deqing Sun, and Ming-Hsuan Yang. Monst3r: A simple approach for estimating geometry in the presence of motion. *ICLR*, 2025.
- [52] Junyi Zhang, Charles Herrmann, Junhwa Hur, Chen Sun, Ming-Hsuan Yang, Forrester Cole, Trevor Darrell, and Deqing Sun. Loger: Long-context geometric reconstruction with hybrid memory. *arXiv preprint arXiv:2603.03269*, 2026.

Supplementary Material

A Additional qualitative results

We have attached additional qualitative results in Figure 6. We can see that G3T achieves better gravity-aligned reconstructions than using the combination of VGGT and GeoCalib (similar to Figure 4 in the main text), even in cases where the input images exhibit challenging roll and pitch angles.

B Dataset pre-processing details

Preparing data for training. As described in Section 3.4, we use five large-scale datasets (MegaDepth [20], Hypersim [26], ARKitScenes [2], DL3DV [22] and TartanAir [46]) for fine-tuning. We use the instructions and code from the DUST3R [45] codebase to download and pre-process data for the MegaDepth and ARKitScenes datasets. We use the instructions and code from the CUT3R [44] codebase to download and pre-process data for the Hypersim, DL3DV and TartanAir datasets.

To fine-tune G3T, we need the ground truth pointmaps to exist in the appropriate gravity-aligned coordinate frames (as defined in Section 3.2). To that end, we require that each scene in each dataset has a world coordinate frame such that the y -axis is aligned with the up direction. This helps us calculate the rotations needed to transform points that exists in the camera frame to the gravity-aligned frame for a given image.

We noticed that the world coordinate frames for the scenes in the MegaDepth dataset are already gravity-aligned such that their y -axis is aligned with the up direction. However, for ARKitScenes, TartanAir and Hypersim, the world coordinate frame of the scenes uses the convention that their z -axis is aligned with the up direction. For these three datasets, we rotate the ground truth data such that the world coordinate frames have their y -axis aligned with the up direction. Finally, we noticed that the DL3DV dataset (downloaded via the CUT3R codebase) is not gravity-aligned. For this, we used the `model_orientation_aligner` module from COLMAP [29, 28] to reorient the ground-truth poses such that the scenes lie in a world coordinate system which is gravity-aligned such that their y -axis is aligned with the up direction.

Preparing data for evaluation. For camera-to-gravity pose estimation and multi-view structure estimation experiments (Section 4.1), we use three datasets (7Scenes [32], NRGBD [1] and ETH3D [30]). For submap-based incremental 3D reconstruction (Section 4.2), we choose sequences from the TUM RGBD dataset [34] for evaluation. Do note that all these datasets were not seen by both VGGT and G3T during training.

We preprocess 7Scenes by following code and instructions from the Spann3R [42] and CUT3R [44] codebases, NRGBD by adapting some code from the CUT3R codebase, and ETH3D by adapting some code from the Mp-SfM [25] codebase. For preprocessing TUM RGBD sequences, we use code from the gradSLAM [15] codebase to synchronize RGB, depth and pose data by timestamps. We then subsample the synchronized TUM RGBD data by selecting every 5th frame.

Similar to the training datasets, we also need to gravity-align the evaluation datasets. To this end, we use the `model_orientation_aligner` module from COLMAP [29, 28] on all of the above datasets to make them gravity aligned (similar to how we processed the training datasets).

On using `model_orientation_aligner`. Prior work has used the `model_orientation_aligner` module from COLMAP to align scenes to canonical orientations. For instance, GeoCalib [39] used it to gravity-align MegaDepth [20] data for evaluation, and MegaScenes [37] dataset used it to make their scenes Manhattan aligned. We follow this precedent to use `model_orientation_aligner` to gravity-align our datasets as well. As a sanity check, Figure 7 visualizes perspective fields on gravity-aligned samples. We can see that up arrows are strongly aligned with the scene’s vertical direction. We additionally evaluate on synthetic datasets that are naturally gravity-aligned (i.e., not processed by `model_orientation_aligner`) in Table 5. The trends observed here are consistent with those in Table 1. This suggests that data aligned with `model_orientation_aligner` is reliable for both training and evaluation.

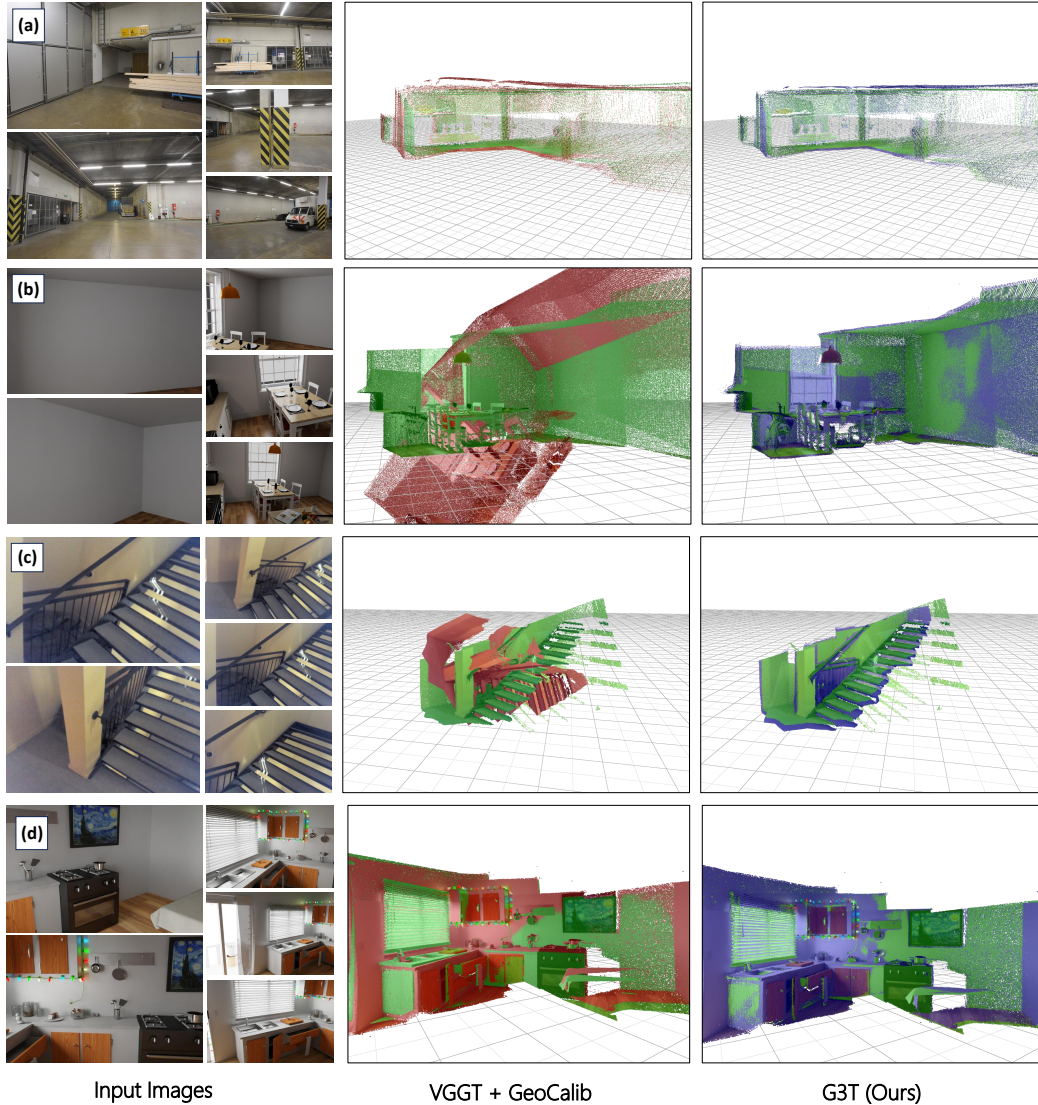


Figure 6: **G3T can consistently place pointmaps in an upright frame.** For the given set of input images, we compare VGGT predictions (**red**) made upright using GeoCalib, and our G3T predictions (**blue**) with ground-truth gravity-aligned pointmaps (**green**). We also render a grid depicting a plane parallel to the ground. We observe that the composition of VGGT and GeoCalib often does not produce sufficiently upright pointmap predictions. In contrast, G3T produces pointmaps that are better aligned along the vertical direction.

Sampling multi-view data. To sample N views from a scene in a dataset, we construct a scene graph data structure that stores the visibility relationships between nodes. For the MegaDepth and ARKitScenes datasets, we build scene graphs using the pair metadata from the DUST3R codebase. For all other training and evaluation datasets, we construct complete scene graphs where every pair of nodes is connected by an edge.

During training, we sample N views from a scene by first randomly selecting an initial node, then randomly selecting $N - 1$ of its neighbors from the scene graph.

For the multi-view pointmap structure experiment (Section 4.1), we adopt a more controlled sampling strategy. We assign a distance value to each edge in the scene graph by summing the rotation difference (computed using Equation 5) and the translation difference (computed using the

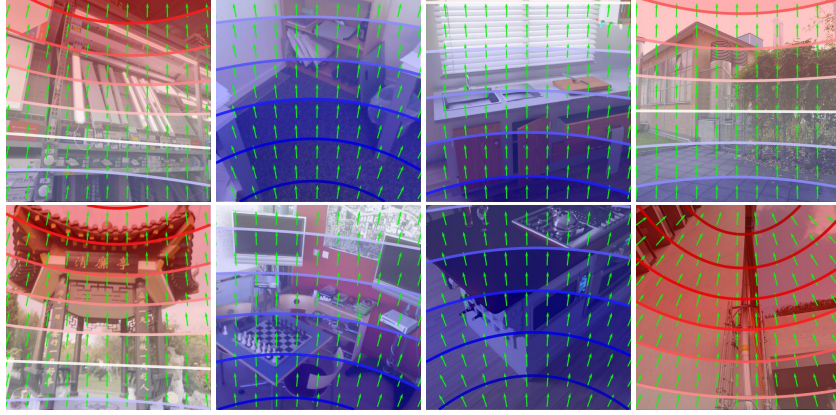


Figure 7: **Visualizing gravity-aligned ground-truth created using model_orientation_aligner.** We visualize perspective fields [13] using gravity-aligned data created using model_orientation_aligner for a few samples from the DL3DV, 7Scenes, NRGBD and ETH3D datasets. For all samples, we can see that the up vectors are aligned with vertical structures present in the scene, thus validating the reliability of using model_orientation_aligner to create ground-truth gravity-aligned data.

Table 5: **Camera to gravity rotation estimation results.** This table mirrors Table 1 from the main paper, but performs evaluation on held out scenes from the Hypersim and TartanAir dataset. These are synthetic datasets which are already gravity aligned (i.e., these datasets were not gravity-aligned using model_orientation_aligner). The trends observed here are consistent with those in Table 1.

Dataset	Model	1V						4V						8V					
		$R_{err} (\downarrow)$		$R_{acc} (\uparrow)$				$R_{err} (\downarrow)$		$R_{acc} (\uparrow)$				$R_{err} (\downarrow)$		$R_{acc} (\uparrow)$			
		Mean	Median	@1°	@2°	@5°		Mean	Median	@1°	@2°	@5°		Mean	Median	@1°	@2°	@5°	
Hypersim	GeoCalib	3.69	1.88	34.40	52.40	89.60		3.77	1.79	35.20	54.00	88.00		3.72	1.73	36.40	55.20	88.80	
	Procrustes	1.49	0.38	87.60	95.20	95.60	0.98	0.41	80.40	92.40	96.80	0.91	0.34	86.00	93.60	97.20			
	Local Head	1.34	0.22	91.20	95.20	96.00	0.58	0.17	94.00	96.00	98.00	0.59	0.17	94.00	95.60	98.40			
TartanAir	GeoCalib	5.68	2.91	19.20	40.00	67.60	5.76	2.94	22.80	39.60	66.80	5.92	2.73	25.60	41.60	65.60			
	Procrustes	1.49	0.89	54.00	76.00	95.60	1.74	1.19	44.80	71.20	94.80	1.67	1.09	47.60	71.60	96.00			
	Local Head	1.52	0.82	56.40	74.80	94.80	1.48	0.99	50.80	76.80	96.80	1.41	0.92	51.60	77.20	97.20			

L^2 norm on mean-normalized translations). We then iteratively sample $N - 1$ neighbor nodes for a randomly selected initial node such that all pairwise distances exceed 0.5, falling back to random neighbor sampling if this fails more than 100 times. This sampling strategy ensures adequate spatial separation between the sampled views.

For the camera-to-gravity pose estimation experiment (Section 4.1), we evaluate performance with 1, 4, and 8 input views. We first sample 8 views following the same procedure as the multi-view pointmap prediction experiment, then construct the 1- and 4-view cases by taking the first 1 and 4 views of each 8-view sample. This ensures the cases are nested subsets, allowing us to cleanly measure the effect of adding more views.

C Additional implementation details

In this section we elaborate on implementation details. Note that we also plan to open-source our codebase.

Local and relative camera heads. In Section 3.2, we briefly introduced the local and relative camera heads in G3T. Here, we provide additional implementation details.

Both heads are derived from the original VGGT camera head implementation with minor modifications. Recall that the original camera head outputs parameters $\mathcal{G} = \{G_i\}$, where $G_i \in \mathbb{R}^9$ (as explained in Section 3.1). In contrast, the local camera head of G3T outputs parameters $\mathcal{G}^l = \{G_i^l\}$ where $G_i^l \in \mathbb{R}^6$, and the relative camera head of G3T outputs parameters $\mathcal{G}^r = \{G_i^r\}$ where $G_i^r \in \mathbb{R}^5$. To accommodate the reduced output dimensionality, we shrink the empty pose token parameter size

Table 6: **Submap-based incremental 3D reconstruction on TUM RGBD: Ambient frame pose metrics (Depthmap unprojection).** This table mirrors Table 3 from the main paper, but uses pointmaps obtained by unprojecting depthmaps with estimated poses. The trends are consistent with those in Table 3.

Method	Metric ($\downarrow$)	fr1/360	fr1/desk	fr1/desk2	fr1/room	fr1/plant	fr1/teddy	fr3/loh ²	fr2/ps ²	fr2/ps2 ²	fr2/ps3 ²
VGGT _D	APE _R	16.790	2.247	1.676	3.962	3.247	4.507	2.844	5.447	8.970	6.775
	APE _t	0.051	0.021	0.037	0.179	0.051	0.087	0.226	0.415	0.398	0.980
	δ_y	0.022	0.004	0.006	0.046	0.020	0.036	0.105	0.209	0.194	0.043
G3T _D	APE _R	18.429	0.735	4.527	2.502	1.740	1.777	4.127	2.776	3.339	6.329
	APE _t	0.049	0.015	0.031	0.150	0.035	0.057	0.178	0.286	0.232	0.211
	δ_y	0.010	0.005	0.014	0.026	0.018	0.017	0.022	0.028	0.032	0.043

Table 7: **Submap-based incremental 3D reconstruction on TUM RGBD: Structure metrics (Depthmap unprojection).** This table mirrors Table 4 from the main paper, but uses pointmaps obtained by unprojecting depthmaps with estimated poses. The trends are consistent with those in Table 4.

Method	Metric	fr1/360	fr1/desk	fr1/desk2	fr1/room	fr1/plant	fr1/teddy	fr3/loh ²	fr2/ps ²	fr2/ps2 ²	fr2/ps3 ²
VGGT _D	ACC ($\downarrow$)	0.053	0.009	0.014	0.051	0.032	0.029	0.047	0.102	0.147	0.447
	COMP ($\downarrow$)	0.065	0.011	0.014	0.047	0.036	0.040	0.034	0.045	0.108	0.382
	NC ($\uparrow$)	0.681	0.616	0.597	0.641	0.607	0.588	0.612	0.670	0.704	0.581
G3T _D	ACC ($\downarrow$)	0.042	0.010	0.014	0.041	0.026	0.022	0.036	0.063	0.073	0.084
	COMP ($\downarrow$)	0.056	0.011	0.014	0.039	0.042	0.038	0.028	0.038	0.068	0.067
	NC ($\uparrow$)	0.692	0.608	0.595	0.661	0.615	0.599	0.632	0.714	0.720	0.740

and the widths of a few linear layers accordingly, then initialize all weights from the pre-trained original camera head.

Additionally, in Section 3.2, we mentioned that the relative camera head output $\mathcal{G}^r = \{G_i^r\}$ contains $q_i^r \in \mathbb{R}^2$ which represents a 1-DoF relative yaw w.r.t. the first frame. Here, q_i^r captures the y and w components of a quaternion, as the x and z components are 0 for a pure 1-DoF rotation along the y -axis.

VGGT-Long global optimization. In Section 3.3, we described the global optimization procedure of VGGT-Long (Equation 2) and our gravity-aligned variant (Equation 4), noting that the norms in both equations are applied to Lie algebra representations of δ_π and δ_{π_y} , respectively. Here we provide further details.

The $\mathfrak{sim}(3)$ Lie algebra is the 7-dimensional tangent space of $\text{Sim}(3)$, with 3 translation, 3 rotation, and 1 scale components. The logarithmic and exponential maps convert between $\text{Sim}(3)$ and $\mathfrak{sim}(3)$ representations. The VGGT-Long [8] codebase uses the PyPose [40] library for these operations, and defines the global optimization objective of Equation 2 in $\mathfrak{sim}(3)$ space. This is advantageous because elements of $\mathfrak{sim}(3)$ are unconstrained, allowing optimization to proceed without constraints.

For our gravity-aligned variant, we instead work with $\text{Sim}_y(3)$ poses and their corresponding 5-dimensional Lie algebra $\mathfrak{sim}_y(3)$. Since $\text{Sim}_y(3)$ restricts rotations to the y -axis, the rotation component of $\mathfrak{sim}_y(3)$ is 1-dimensional. We modify the VGGT-Long codebase to support the gravity-aligned global optimization of Equation 4.

Configuring submap-based incremental reconstruction. Here, we discuss the hyperparameters used for the submap-based incremental 3D reconstruction experiments in Section 4.2. As described in Section 3.3, VGGT-Long processes long sequences by breaking them into overlapping chunks. We set the maximum chunk size to 25 frames, with 7 overlapping frames between adjacent chunks. VGGT-Long also forms chunks across loop closure detections for use in the optimization process, controlled by a loop chunk size hyperparameter; in general, each loop closure chunk spans twice this value. We set the loop chunk size to 3.

We additionally measure the effect of number of overlapping frames, as more overlap can intuitively lead to more stable chunk alignment estimates. Specifically, we repeat the experiments of Section 4.2 with 3 and 15 overlapping frames (instead of 7), keeping all other hyperparameters fixed. Figure 8 plots pose metrics (APE_R, APE_t) and structure metrics (ACC, COMP) as a function of number

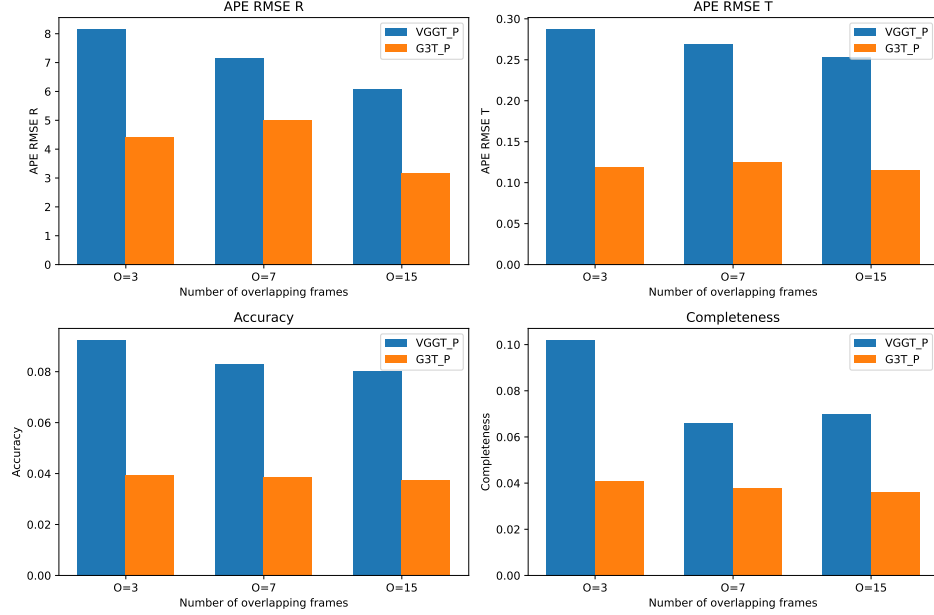


Figure 8: **Effect of number of overlapping frames in submap-based incremental reconstruction.** Here, we plot pose metrics (APE_R , APE_t) and structure metrics (ACC, COMP) as a function of number of overlapping frames, averaged across the 10 sequences from Tables 3 and 4 (lower is better for all metrics). While performance generally improves with more overlapping frames, we observe that G3T has a consistent lead across all settings, with the largest margin when the number of overlapping frames is small.

of overlapping frames, averaged across the 10 sequences from Tables 3 and 4 (lower is better for the plotted metrics). Performance generally improves with more overlapping frames. Nevertheless, G3T_P maintains a consistent lead across all settings, with the largest margin when the number of overlapping frames is small.

GA-Procrustes. For Procrustes, we use the `rigid_points_registration` function from the open-source RoMA library [3]. For GA-Procrustes, we implement a modified version incorporating the changes described in Section 3.3.

Camera-to-gravity pose estimation using Procrustes. In Section 4.1, we briefly introduced our algorithm to compute the camera-to-gravity pose given predictions from G3T and VGGT using Procrustes alignment within a RANSAC loop. Here, we elaborate on the implementation details.

Given a set of N images $\mathcal{I}$, we predict a set of gravity-aligned pointmaps $\mathcal{X}^{G_1}$ using G3T and a set of camera-aligned pointmaps $\mathcal{X}^{C_1}$ using VGGT, along with their associated confidence maps Ψ^{G_1} and Ψ^{C_1} . To prepare these predictions for robust alignment, we first pre-process the data as follows:

We create a joint confidence map Ψ by element-wise multiplying the logarithms of the confidence maps from each model: $\Psi = \log \Psi^{G_1} \odot \log \Psi^{C_1}$. This ensures that regions with high confidence in both models have higher Ψ values, while regions with low confidence in either model have lower Ψ values. We then select the top 50% of points based on their confidence values and independently scale-normalize them, where the scale is defined as the median distance of points to the pointmap center (the median of all points). We denote the resultant pointmaps as $\hat{\mathcal{X}}^{G_1}$ and $\hat{\mathcal{X}}^{C_1}$.

We then run a RANSAC loop for 5,000 iterations. In each iteration, we randomly sample 50,000 corresponding points from $\hat{\mathcal{X}}^{G_1}$ and $\hat{\mathcal{X}}^{C_1}$, and use the `rigid_vectors_registration` function from the open-source RoMA library [3] to estimate the rotation R and scale s (we discard the scale). We apply the predicted rotation matrix to transform $\hat{\mathcal{X}}^{C_1}$ and compare it with $\hat{\mathcal{X}}^{G_1}$ to count the

²We shorten `long_office_household` to `loh` and `pioneer_slam` to `ps` for brevity.

number of inliers (defined as points with L^2 error less than 0.05). Finally, we return the rotation matrix R with the highest number of inliers.

On using GeoCalib. Recall that VGGT outputs pointmaps in a camera coordinate frame. To create gravity-aligned pointmaps in this case, we use GeoCalib [39] to compute the camera to gravity transform. While GeoCalib estimates gravity from a single image, it also supports joint optimization across multiple images. For all experiments involving GeoCalib, we use the multi-image optimization mode.

Pointmap visualizations. Several figures in this paper compare the upright alignment of gravity-aligned predicted pointmaps against ground-truth pointmaps. For each visualization, we use pointmaps extracted directly from the point head (i.e., VGGT_P and G3T_P). Since VGGT does not guarantee upright outputs, we apply GeoCalib with multi-image optimization to make its pointmaps gravity-aligned. We then apply GA-Procrustes to align each predicted pointmap to the ground-truth, producing a visual comparison that highlights deviations from uprightness.